

TP10 : Algorithmes

I. Complexité temporelle et spatiale

1) La complexité temporelle

La complexité temporelle d'un algorithme est une mesure du temps que prend un algorithme pour s'exécuter en fonction de la taille de l'entrée. Elle est généralement exprimée à l'aide de la notation Big O, qui décrit le comportement asymptotique de l'algorithme.

2) La complexité spatiale

La complexité spatiale d'un algorithme est une mesure de l'espace mémoire que requiert un algorithme pour s'exécuter en fonction de la taille de l'entrée. Elle tient compte de la mémoire supplémentaire utilisée par l'algorithme (en dehors de l'entrée elle-même).

II. Notation Big O

La notation Big O décrit le comportement limite d'une fonction lorsque l'argument tend vers l'infini. En algorithmique, elle permet de classer les algorithmes selon leur croissance en temps ou en espace.

Classes de complexité courantes :

- $O(1)$: Temps constant (accès à un élément d'un tableau)
- $O(\log n)$: Temps logarithmique (recherche binaire)
- $O(n)$: Temps linéaire (parcours d'un tableau)
- $O(n \log n)$: Temps quasi-linéaire (tri fusion, tri rapide)
- $O(n^2)$: Temps quadratique (tri bulles, tri sélection)
- $O(2^n)$: Temps exponentiel (problème du voyageur de commerce)

III. Algorithmes de tri

Voici une liste exhaustive des algorithmes de tri :

A. Tri par Sélection (Selection Sort)

Principe : Rechercher le plus petit élément et le placer à sa position finale.

De manière concrète faire une boucle **i** sur chaque élément des données d'entrée, initialement dans cette boucle affecter l'indice minimale l_{min} à **i**. et pour chaque indice **i**, construire une nouvelle boucle **j** sur les éléments restant à partir de **i** et dans l'ordre des indices, ce qui fait une boucle imbriquée. On détermine ensuite sur

la boucle imbriquée l'indice de l'élément le plus petit parmi tous les éléments à la suite de l'indice i . Au sortir de la deuxième boucle (boucle imbriquée), on permute les données $A[i]$ et $A[iMin]$ entre indices $iMin$ du minimum et i . une fois la boucle i terminer nos données d'entrées sont ordonnées

Algorithme C++

```
#include<iostream>
using namespace std;

void selectionSort(int A[], int n){
    for (int i=0; i<n-1; i++){ // we need to do n-2 passes
        int iMin=i; // ith position: elements from i till n-1

        for (int j=i+1; j<n; j++){
            if (A[j]<A[iMin]) iMin=j;
        }
        int temp = A[i];
        A[i]=A[iMin];
        A[iMin]=temp;
    }
}

int main(){
    int x,y;
    int A[] = {2,7,4,1,5,3};
    selectionSort(A,6);
    for (int i=0; i<6; i++) cout<<A[i]<<" ";
    cin>>x;
    cin>>y;
    cout<<"value of x:"<<x<<"\n value of y : "<<y;
}
```

Complexité : Temps= $(n-1)+(n-2)+(n-3)+.....+2+1=n(n-1)/2= O(n^2)$, Espace= $O(1)$ =car la taille des données d'entrées stockées ne change pas au cours de toutes les opérations de trie

B. Tri par Insertion (Insertion Sort)

Principe : Insérer chaque élément à sa place dans la partie déjà triée.

De manière concrète, construire une boucle qui parcourt chaque élément d'indice i et pour chacun d'eux, faire une boucle while qui la compare avec les éléments qui la précèdent, dans le cas ou elle plus petite que l'un de ces élément, procéder à la permutation.

Complexité : Temps= $O(n^2)$ (meilleur cas $O(n)$ pour des données d'entrées ordonné), Espace = $O(1)$

Ce Tri est meilleur que le cas de selection sort car plus les données se rapproche d'un tableau ordonné et moins on a d'opération à faire

Pseudo Algorithme

```
1 InsertionSort(A)
2   pour i de 1 à n-1
3       clé = A[i]
4       j = i-1
5       tant que j >= 0 et A[j] > clé
6           A[j+1] = A[j]
7           j = j-1
8       A[j+1] = clé
```

C. Tri à Bulles (Bubble Sort)

Principe : Comparer et échanger les éléments adjacents.

De manière concrète, on parcourt les données d'entrées en switchant les éléments adjacents (successif) ceci dans le cas où le premier est supérieur au suivant. et on continue ainsi jusqu'au dernier élément qui sera forcément le plus grand. Ensuite on recommence; mais cette fois sur n-1 élément et ainsi de suite jusqu'au 2 derniers indices.

Complexité : Temps= $O(n^2)$, Espace= $O(1)$

Pseudo Algorithme

```
1 BubbleSort(A)
2   pour i de 0 à n-2
3       pour j de 0 à n-2-i
4           si A[j] > A[j+1]
5               échanger A[j] et A[j+1]
```

D. Tri Rapide (QuickSort)

Principe : Diviser pour régner avec un pivot.

De manière concrète, on partitionne le tableau en deux parties, avec comme point de division l'élément pivot. Tous les éléments plus petits que le pivot sont placés à gauche du pivot constitue notre premier tableau de partitions; puis tous les éléments plus grands que le pivot sont placés à droite du pivot et constituent notre deuxième tableau de partitions. si les deux tableaux ne sont également pas ordonné, on leur applique également la fonction de trie Quicksort qui dans ce cas s'appelle elle

même(principe de récursivité), et donc vas également sur chaque partition appliquer de nouveau la partitions et ensuite si pas d'ordre, l'appel récursif est déclenchée une fois de plus ceci jusqu'à ce qu'on tombe sur des partitions ordonnées et à partir de ce moment l'appel récursif est stopper.

Complexité : **Temps**= $O(n \log n)$ moyen, $O(n^2)$ pire cas, **Espace**= $O(\log n)$

Algorithme C++

```
1 // quicksort.cpp
2 #include<iostream>
3 using namespace std;
4
5 int partition(int *A, int start, int end){
6     int pivot=A[end];
7     int partitionIndex=start; // set partition index as start
8     for (int i=start; i<end; i++){
9         if (A[i]<=pivot){
10             swap(A[i], A[partitionIndex]); // swap if element
11             partitionIndex++;
12         }
13     }
14     swap(A[partitionIndex], A[end]); // swap pivot with element
15     return partitionIndex;
16 }
17
18 void quicksort(int *A, int start, int end){
19     if (start<end){
20         int partitionIndex = partition(A, start, end);
21         quicksort(A, start, partitionIndex-1);
22         quicksort(A, partitionIndex+1, end);
23     }
24 }
25
26 int main(){
27     int A[]={7,2,1,6,8,5,3,4};
28     quicksort(A,0,7);
29     for (int i=0; i<8; i++) cout<<A[i]<<" ";
30 }
```

E. Tri Fusion (Merge Sort)

Principe : Diviser le tableau en deux, trier récursivement, fusionner.

De manière concrète: Diviser le tableau en deux, si les élément des deux tableaux obtenus sont ordonnés alors appliqué l'algorithme Merge qui permet de ranger facilement 2 tableaux déjà ordonnés.

sauf que n'étant pas sûr que les 2 tableaux sont ordonnés, il faut également les ordonner avant de passer à Merge qui va ainsi les ranger.

Pour cela on va appliquer notre même fonction de trie MergeSort sur les deux Tableaux, ce qui entraîne que notre fonction s'appelle encore elle-même d'où la récursivité.

on constate donc que tant qu'on n'est pas sûre de l'ordre des 2 tableaux obtenu, la fonction est obligé de s'appeler elle même jusqu'à ce qu'on arrive sur 2 tableaux sur lequel on est sure qu'il sont ordonné et ce cas correspond à la situation ou les 2 tableaux ont chacun un élément. ce point correspond au point d'arrêt de la récursivité

Complexité : Temps= $O(n \log n)$, **Espace** = $O(n)$

Pseudo Algorithme

```
1 MergeSort(A, gauche, droite)
2   si gauche < droite
3     milieu = (gauche + droite) / 2
4     MergeSort(A, gauche, milieu)
5     MergeSort(A, milieu+1, droite)
6     Fusionner(A, gauche, milieu, droite)
```

F. Tri par Tas (Heap Sort)

Principe : Construire un tas, extraire les éléments dans l'ordre.

Complexité : Temps= $O(n \log n)$, **Espace** = $O(1)$

Pseudo Algorithme

```
1 HeapSort(A)
2   construireMaxHeap(A)
3   pour i de n-1 à 1
4     échanger A[0] et A[i]
5     tailleTas = i
6     maxHeapify(A, 0, tailleTas)
```

G. Tri par Comptage (Counting Sort)

Principe : Compter les occurrences de chaque valeur.

Complexité : Temps= $O(n+k)$, **Espace**= $O(n+k)$

Pseudo Algorithme

```

1 CountingSort(A, k)
2   créer C[0..k] initialisé à 0
3   pour i de 0 à n-1
4     C[A[i]] = C[A[i]] + 1
5   pour i de 1 à k
6     C[i] = C[i] + C[i-1]
7   créer B[0..n-1]
8   pour i de n-1 à 0
9     B[C[A[i]]-1] = A[i]
10    C[A[i]] = C[A[i]] - 1

```

H. Tri par Base (Radix Sort)

Principe : Trier chiffre par chiffre (LSD ou MSD).

Complexité : Temps = $O(d(n+k))$, Espace = $O(n+k)$

Pseudo Algorithme

```

1 RadixSort(A, d)
2   pour i de 1 à d
3     utiliser un tri stable sur le i-ème chiffre

```

I. Tri par Panier (Bucket Sort)

Principe : Distribuer dans des buckets, trier chaque bucket.

Complexité : Temps = $O(n)$ moyen, $O(n^2)$ pire cas, Espace = $O(n)$

Pseudo Algorithme

```

1 BucketSort(A)
2   créer n buckets vides
3   pour i de 0 à n-1
4     insérer A[i] dans bucket[floor(n*A[i])]
5   pour i de 0 à n-1
6     trier bucket[i]
7   concaténer les buckets

```

J. Autres algorithmes :

- Tri Cocktail (Cocktail Shaker Sort)
- Tri par Peigne (Comb Sort)
- Tri de Shell (Shell Sort)
- Tri de Gnome (Gnome Sort)
- Tri par Cycle (Cycle Sort)

- Tri par Échange (Exchange Sort)
- Tri par Pigeonhole (Pigeonhole Sort)
- Timsort (hybride merge/insertion)
- Introsort (hybride quick/heap/insertion)
- Smoothsort
- BogoSort (aléatoire)
- StoogeSort

Tableau Comparatif des Algorithmes de Tri

Algorithme	Temps Moyen	Temps Pire Cas	Espace	Stable	In-place
Tri Sélection	$O(n^2)$	$O(n^2)$	$O(1)$	Non	Oui
Tri Insertion	$O(n^2)$	$O(n^2)$	$O(1)$	Oui	Oui
Tri Bulles	$O(n^2)$	$O(n^2)$	$O(1)$	Oui	Oui
QuickSort	$O(n \log n)$	$O(n^2)$	$O(\log n)$	Non	Oui
Tri Fusion	$O(n \log n)$	$O(n \log n)$	$O(n)$	Oui	Non
Tri Tas	$O(n \log n)$	$O(n \log n)$	$O(1)$	Non	Oui

Tri Comptage	$O(n+k)$	$O(n+k)$	$O(n+k)$	Oui	Non
-----------------	----------	----------	----------	-----	-----

Tri Base	$O(d(n+k))$	$O(d(n+k))$	$O(n+k)$	Oui	Non
----------	-------------	-------------	----------	-----	-----

Tri Panier	$O(n+k)$	$O(n^2)$	$O(n)$	Oui	Non
------------	----------	----------	--------	-----	-----